B9CY108 Report

Repository link: https://github.com/Calibobo1994/ADVANCED-PROGRAMMING
The report is broken down into 4 tasks and each states the work done in each task.

Task 1:
The abstract Staff class stores each staff member's name, ID and department, with validation to prevent blank values and ensure IDs are trimmed and converted to uppercase. Doctor, Nurse and AdministrativeStaff inherit from it, implementing their own PerformDuty() methods and extending GetDetails() with role-specific information. StaffManager manages the staff list, prevents duplicate IDs and supports case-insensitive searches. InputPrompt.ReadRequired prevents empty input, while Program.cs provides a six-option menu for adding staff, listing, searching and exiting. The project also includes 10 xUnit tests covering searches, case-insensitivity, duplicate IDs, null values, insertion order, setter validation and role-specific duties.

Task 2:
The Student class stores each student's ID, name and module marks in a dictionary, calculates their average and uses a static GetGrade method to assign grades (B+ for 70 and above, B for 65, C for 55, D for 45, E for 40, and F below 40). StudentRepository provides 20 sample students across five modules and supports student lookups by ID. The console application repeatedly prompts for a student ID and, when a match is found, displays a formatted table of module marks and corresponding grades, followed by the overall average. It also handles unknown IDs and prevents blank input.

Task 3:
ThreadingTCPServer handles incoming client connections, accepting one newline-terminated JSON request per connection. Its check_request method validates required fields, non-empty text, contact numbers (7–15 digits, supporting "+", spaces and hyphens), email formats and membership periods (6 or 12 months). Valid requests are saved to the database and assigned unique registration numbers in the "LIB-000001" format. The client performs the same validation before sending requests to catch invalid data early. The database schema supports safe repeated creation and includes a CHECK constraint for membership periods. Both client and server also handle rejected requests, connection timeouts and database errors with clear error messages.

Task 4:

Que4.py fetches the Dublin district list from Wikipedia, identifies headings matching the "Dublin N (D#)" format and extracts linked place names from each district's "includes…" sentence, cleaning footnotes and irrelevant phrases such as "and contains" and "Notable". The extracted data is saved as a sorted JSON file grouped by district and a normalised two-column CSV. The script then reads both files back and prints the resulting table to verify the output. The final dataset contains 22 districts and 166 areas, excluding D19, D21 and D23 because those districts do not exist.